

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

2. Primeros pasos en Haskell

Empecemos por los comentarios

```
-- comentario hasta fin de línea  
{- comentario flexible -}
```

Declaraciones y definiciones

```
doblemas1 :: Int -> Int  
doblemas1 x = 2 * x + 1  
  
res :: Int  
res = doblemas1 33
```

- La declaración de tipo es opcional, pero conveniente.
 - Haskell sabe inferir tipos.

Normas sobre identificadores

- Minúscula inicial: variables, funciones y valores.
- Mayúscula inicial: tipos.

Operadores

```
esBisiesto :: Int -> Bool  
esBisiesto x = ((mod x 4 == 0) && (mod x 100 /= 0)) || (mod x 400 == 0)  
  
resto4 :: Int -> Int  
resto4 x = x `mod` 4 -- función a operador  
  
mas1 :: Int -> Int  
mas1 x = (+) x 1 -- operador a función
```

Polimorfismo

- `dobleMas1 4.0` es un error de tipado.
- Declarar ahora `dobleMas1 :: Float -> Float` es un error, porque `dobleMas1` ya está declarado.

```
dobleMas1' :: Num a => a -> a
dobleMas1' x = 2 * x + 1
```

- Se dice que `dobleMas1'` es una función polimórfica.
- Aquí, `a` es una variable de tipo. Y se nombra con minúscula inicial.
- La comilla simple se puede usar en identificadores.
- `Num a` es una restricción de tipo.
- `Num` es una clase de tipos.
 - La definición de `Num` dice que pertenecen a esta clase los tipos que tienen definidos los operadores `*` y `+` (y otras cosas).
- Es decir, `dobleMas1'` tiene tipo `a -> a` para cualquier tipo `a` que pertenezca a la clase `Num`.
- La definición de la función debe valer igual para todos los tipos que cumplan las restricciones.
- También los valores pueden ser polimórficos
 - `[] :: [a]`
 - `3 :: Num a => a`

Aplicación de funciones

- La aplicación de funciones tiene máxima prioridad:
 - `1 + dobleMas1 4 * 5` $\rightsquigarrow$ 46
- Podemos usar paréntesis:
 - `1 + dobleMas1 (4 * 5)` $\rightsquigarrow$ 42
- El operador `$` hace que la aplicación de la función tenga mínima prioridad:
 - `dobleMas1 $ 4 * 5 + 1` $\rightsquigarrow$ 43

Algunos tipos básicos

- `Bool` (contiene los valores `True` y `False`)
- `Int`
- `Float`
- `Char` (`'z'`)
- `String` (`"xyz"`)

Notación matemática y Haskell

matemáticas	Haskell
$f(x)$	<code>f x</code>
$f(x, y)$	<code>f x y</code>
$f(g(x))$	<code>f (g x)</code>
$f(x, g(y))$	<code>f x (g y)</code>
$f(x) \cdot g(y)$	<code>f x * g y</code>

Más cosas

- Haskell es *perezoso*: `False && div 1 0 == 7` $\rightsquigarrow$ `False`
- Haskell tiene *tipado estático*: `div 1 0 == 'a'` es un error de tipo.
- Un programa es una serie de definiciones de funciones (no instrucciones).
- Computar es evaluar expresiones (no ejecutar instrucciones).

Algunas funciones predefinidas

- <https://hackage.haskell.org/package/base/docs/Prelude.html>
- <https://hoogle.haskell.org/>

- Aritmética y números:

```
- (+), (-), (*) :: Num a => a -> a -> a
- (/) :: Fractional a => a -> a -> a
- div, mod :: Integral a => a -> a -> a
- (^) :: (Num a, Integral b) => a -> b -> a (potencia)
- abs :: Num a => a -> a
- floor, ceiling :: (RealFrac a, Integral b) => a -> b
- log :: Floating a => a -> a
```

- Comparación y orden:

```
- (==), (/=) :: Eq a => a -> a -> Bool
- (<), (>), (<=), (>=) :: Ord a => a -> a -> Bool
- max, min :: Ord a => a -> a -> a
```

- Booleanos:

```
- (&&), (||) :: Bool -> Bool -> Bool
- not :: Bool -> Bool
```

Ejercicios

1. Declara y define la función `celsiusAFahrenheit` mediante la fórmula $F = \frac{9}{5}C + 32$.
2. Declara y define una función que devuelva el último dígito (el de las unidades) de un número natural. Otra que devuelva el dígito de las centenas.
3. Escribe todas las definiciones posibles para funciones con estos tipos (sin restricciones de tipos):
 - `f :: a -> Bool`
 - `g :: a -> a`
 - `h :: a -> a -> a`